Simulation of Engineering System 3 (ENG3036)

Assignment Report Part 2

Student Name: Fatimah Hasan **Student Number: 2888172h**

Assignment Topic: Turning Control System for a Wheeled Rover

2.0 Introduction for Part 2

The initial proportional controller designed in Part 1 ($G_C = 11.6$) produced a highly oscillatory response with significant overshoot and a long settling time. The purpose of Part 2 is to redesign the controller to improve performance, and to test the system under more realistic operating conditions by varying the forward velocity.

This report covers three stages:

- **Gc tuning:** Investigating the effect of varying the proportional gain G_C to reduce oscillation and selecting the optimal value.
- **PI controller:** Adding an integral term to form a PI controller, implemented using hand coded Euler integration within the RK4 loop, to eliminate steady state error.
- **Interpolation:** Using Newton's Divided Difference method to model the time varying forward velocity V_T from tabulated data, and analysing its effect on the rover's turning response.

2.1 Gc Control Gain Variation Results & Analysis

The initial proportional controller uses $\Delta V = G_C \Delta\psi$. The original gain $G_C = 11.6$ produced excessive oscillation. To find a better value, the simulation was run with three values of G_C: 0.7, 5 and 11.6.

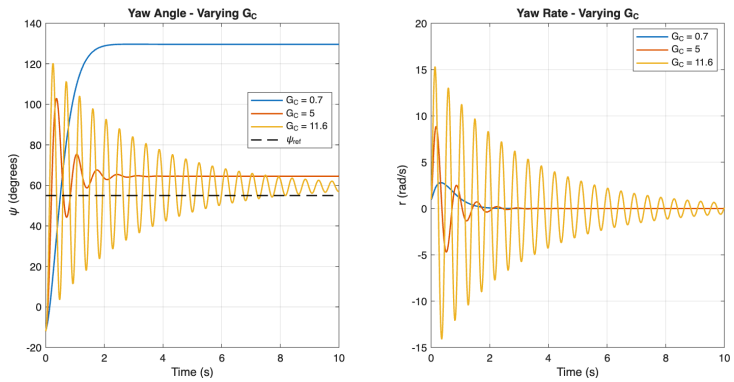


Figure 1: Yaw angle and yaw rate responses for three values of proportional gain G_C. Increasing G_C reduces steady state error but introduces oscillation; $G_C = 5$ offers the best compromise between speed and stability

$G_C = 0.7$ (blue): The low gain eliminates oscillation entirely, producing a smooth first order like response. However, the system settles at approximately 130 — far above the 55 reference. The gain is too low to provide sufficient control authority, resulting in a large steady state error.

$G_C = 11.6$ (yellow): The original Part 1 gain produces sustained oscillations that persist beyond 10 seconds. The system is stable but heavily underdamped, with overshoot exceeding 100%. This confirms the controller is too aggressive.

$G_C = 5$ (red): This value provides the best compromise. The response has moderate overshoot (≈ 20), settles within approximately 4 seconds, and reaches a final heading of ≈ 65. Although a steady state error of ≈ 10 remains (inherent to proportional only control), the transient performance is significantly improved.

$G_C = 5$ was selected as the optimal proportional gain for the subsequent PI controller design.

2.2 Integral Implementation, K_I Tuning, Results & Analysis

To eliminate the steady state error, an integral term was added to form a PI controller (Eq. 2 from the specification):

$$\Delta V = G_C \left(1 + \frac{K_I}{s}\right) \Delta\psi$$

In the time domain this becomes:

$$\Delta V = G_C \left(\Delta\psi + K_I \int_0^t \Delta\psi \, d\tau\right)$$

The integral was implemented by adding a 10th state variable to the system: $x_{10} = \int_0^t \Delta\psi \, d\tau$, with derivative $\dot{x}_{10} = \Delta\psi$ and initial condition $x_{10}(0) = 0$. This state is integrated within the RK4 loop alongside the other nine states no built in MATLAB functions were used.

Using $G_C = 5$, three values of K_I were tested: 0, 0.5 and 2.

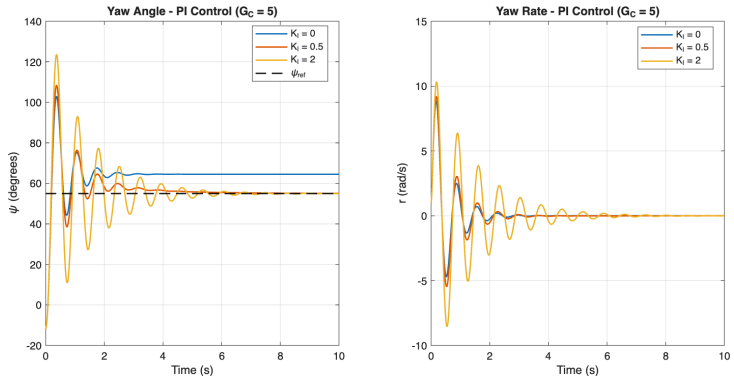


Figure 2: Yaw angle and yaw rate responses for the PI controller ($G_C = 5$) with varying integral gain K_I.

$K_I = 0$ (blue): Equivalent to proportional only control. The heading settles at ≈ 65 with a 10 steady state error, confirming the need for integral action.

$K_I = 0.5$ (red): The integral term drives the steady-state error to zero — the heading settles exactly at 55. The transient response remains well damped with acceptable overshoot. This is the optimal value.

$K_I = 2$ (yellow): The higher integral gain introduces additional oscillation and overshoot. While the steady state error is still eliminated, the transient performance is degraded.

The final controller parameters selected are $G_C = 5$ and $K_I = 0.5$.

2.3 Interpolation Calculation

Newton's Divided Difference method was used to construct an interpolating polynomial for the forward velocity $V_T(t)$ from the data in Table 1 (Appendix B):

t (s)	0	1.2	3.2	6	9	10
V_T (m/s)	0.72	0.8	1.5	0.6	0.4	0.2

The divided difference table is constructed as follows:

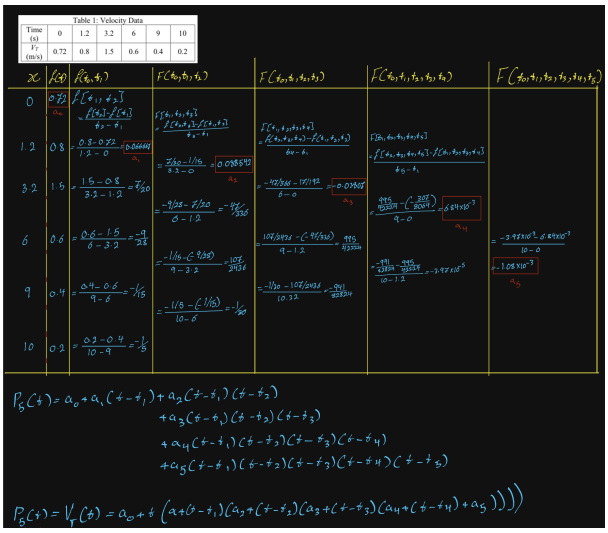


Figure 3: Hand calculation of Newton's Divided Difference table showing the zeroth through fifth order divided differences, with the resulting coefficients a_0 to a_5 highlighted.

2.4 Interpolation Implementation, Results & Analysis

The interpolating polynomial was implemented in MATLAB as a function `newtonInterp.m` using Horner's nested form. At each time step of the RK4 simulation, V_T is evaluated from the polynomial rather than being held constant.

The interpolated polynomial passes through all six data points exactly, confirming correct implementation. The velocity rises from 0.72 m/s to a peak of 1.5 m/s at $t = 3.2$ s, then decreases to 0.2 m/s by $t = 10$ s.

The PI controller ($G_C = 5$, $K_I = 0.5$) was then run with this time varying V_T:

Results:

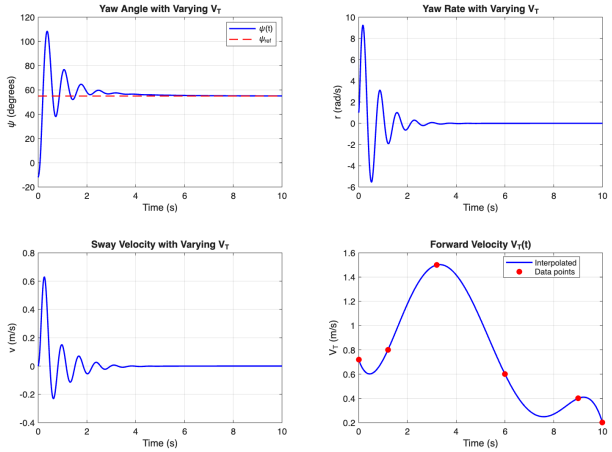


Figure 4: System response with forward velocity $V_T(t)$ varying according to the Newton's divided difference interpolating polynomial fitted to the data points in Table 1.The interpolated $V_T(t)$ profile (bottom right) rises to a peak of 1.5 m/s at $t \approx 3.2$ s before decaying to 0.2 m/s.

Effect on yaw angle: The heading still settles to 55, confirming that the integral term successfully eliminates steady state error even with a time varying plant. However, the transient response shows additional oscillation during the period when V_T is changing rapidly ($t = 0$–4 s).

Effect on dynamics: The variation in V_T directly affects equations (7) and (8), as V_T appears as a coefficient coupling the sway and yaw dynamics. When V_T is high (≈ 1.5 m/s around $t = 3$ s), the coupling is stronger, producing more dynamic interaction. As V_T decreases after $t = 6$ s, the system becomes more decoupled and settles.

Robustness of PI controller: Despite the time varying plant, the PI controller maintains zero steady state error. This demonstrates that the integral action provides robustness against changes in the forward velocity.

MatLab Code:

```m
% ============================================================
%  PART 2 - STEP 1: VARY Gc (Proportional Gain Study)
% ============================================================

Gc_values = [0.7, 5, 11.6];

figure('Name','Part 2 Step 1: Gc Variation','Color','w',...
       'Position',[50 50 1100 500]);

% Store results for analysis
results_Gc = struct();

for idx = 1:length(Gc_values)
    p_temp = p;
    p_temp.GC = Gc_values(idx);

    % RK4 integration
    X = zeros(9, N);
    X(:,1) = x0;
    for n = 1:N-1
        xn = X(:,n);
        k1 = roverDerivatives(t(n),        xn,           p_temp);
        k2 = roverDerivatives(t(n)+dt/2,   xn+(dt/2)*k1,  p_temp);
        k3 = roverDerivatives(t(n)+dt/2,   xn+(dt/2)*k2,  p_temp);
        k4 = roverDerivatives(t(n)+dt,     xn+dt*k3,      p_temp);
        X(:,n+1) = xn + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
    end

    psi_deg = X(9,:)*180/pi;

    % Plot yaw angle
    subplot(1,2,1)
    plot(t, psi_deg, 'LineWidth', 1.3, ...
         'DisplayName', ['G_C = ' num2str(Gc_values(idx))]); hold on;

    % Plot yaw rate
    subplot(1,2,2)
    plot(t, X(8,:), 'LineWidth', 1.3, ...
         'DisplayName', ['G_C = ' num2str(Gc_values(idx))]); hold on;

    % Print metrics
    overshoot = max(psi_deg) - 55;
    ss_error  = 55 - psi_deg(end);
    fprintf('Gc = %5.1f | Overshoot = %6.1f deg | SS Error = %5.2f deg | Final = %5.1f deg\n',...
            Gc_values(idx), overshoot, ss_error, psi_deg(end));
end

subplot(1,2,1)
plot(t, 55*ones(size(t)), 'k--', 'LineWidth', 1.2, 'DisplayName', '\psi_{ref}');
grid on; xlabel('Time (s)'); ylabel('\psi (degrees)');
title('Yaw Angle - Varying G_C'); legend('Location','best');

subplot(1,2,2)
grid on; xlabel('Time (s)'); ylabel('r (rad/s)');
title('Yaw Rate - Varying G_C'); legend('Location','best');

% ---- Identify best Gc ----
% From the results, select the Gc that gives good settling with
% acceptable overshoot. Typically a lower Gc (e.g. 2-5) reduces
% oscillation significantly.
fprintf('\n>> Select the best Gc from the results above.\n');
fprintf('>> A good choice balances low overshoot with fast settling.\n\n');

best_Gc = 5;  % <-- UPDATE THIS based on your results
fprintf('Selected best Gc = %.1f for PI controller design.\n\n', best_Gc);
```

```m
%  PART 2 - STEP 2: PI CONTROLLER
% ============================================================
% PI control law (Eq 2 from Part 2 spec):
%   DeltaV = Gc * (1 + KI/s) * Delta_psi
% In time domain:
%   DeltaV = Gc * (Delta_psi + KI * integral(Delta_psi dt))
%
% The integral is computed using Euler integration within the RK4 loop.
% State vector is extended to 10 states: [original 9; integral_error]

KI_values = [0, 0.5, 2];

figure('Name','Part 2 Step 2: PI Control - KI Variation','Color','w',...
       'Position',[50 50 1100 500]);

for idx = 1:length(KI_values)
    p_pi = p;
    p_pi.GC = best_Gc;
    p_pi.KI = KI_values(idx);

    % Extended state: x(10) = integral of heading error
    X = zeros(10, N);
    X(1:9,1) = x0;
    X(10,1)  = 0;  % integral starts at zero

    for n = 1:N-1
        xn = X(:,n);
        k1 = roverDerivatives_PI(t(n),        xn,           p_pi);
        k2 = roverDerivatives_PI(t(n)+dt/2,   xn+(dt/2)*k1,  p_pi);
        k3 = roverDerivatives_PI(t(n)+dt/2,   xn+(dt/2)*k2,  p_pi);
        k4 = roverDerivatives_PI(t(n)+dt,     xn+dt*k3,      p_pi);
        X(:,n+1) = xn + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
    end

    psi_deg = X(9,:)*180/pi;

    subplot(1,2,1)
    plot(t, psi_deg, 'LineWidth', 1.3, ...
         'DisplayName', ['K_I = ' num2str(KI_values(idx))]); hold on;

    subplot(1,2,2)
    plot(t, X(8,:), 'LineWidth', 1.3, ...
         'DisplayName', ['K_I = ' num2str(KI_values(idx))]); hold on;

    overshoot = max(psi_deg) - 55;
    ss_error  = 55 - psi_deg(end);
    fprintf('KI = %4.1f | Overshoot = %6.1f deg | SS Error = %5.2f deg | Final = %5.1f deg\n',...
            KI_values(idx), overshoot, ss_error, psi_deg(end));
end

subplot(1,2,1)
plot(t, 55*ones(size(t)), 'k--', 'LineWidth', 1.2, 'DisplayName', '\psi_{ref}');
grid on; xlabel('Time (s)'); ylabel('\psi (degrees)');
title(['Yaw Angle - PI Control (G_C = ' num2str(best_Gc) ')']);
legend('Location','best');

subplot(1,2,2)
grid on; xlabel('Time (s)'); ylabel('r (rad/s)');
title(['Yaw Rate - PI Control (G_C = ' num2str(best_Gc) ')']);
legend('Location','best');

% Select best KI
best_KI = 0.5;  % <-- UPDATE THIS based on your results
fprintf('\nSelected best KI = %.1f\n\n', best_KI);
```

```m
% ============================================================
%  PART 2 - STEP 3: NEWTON'S DIVIDED DIFFERENCE INTERPOLATION
% ============================================================
% Data from Table 1 (Appendix B):
%   Time (s):  0    1.2   3.2   6     9     10
%   VT (m/s):  0.72 0.8   1.5   0.6   0.4   0.2

t_data  = [0, 1.2, 3.2, 6, 9, 10];
VT_data = [0.72, 0.8, 1.5, 0.6, 0.4, 0.2];
n_pts   = length(t_data);

% ---- Build Newton's Divided Difference Table ----
% dd(i,j) = f[x_i, x_{i+1}, ..., x_{i+j}]
dd = zeros(n_pts, n_pts);
dd(:,1) = VT_data';  % zeroth-order differences = data values

for j = 2:n_pts
    for i = 1:(n_pts - j + 1)
        dd(i,j) = (dd(i+1,j-1) - dd(i,j-1)) / (t_data(i+j-1) - t_data(i));
    end
end

fprintf('Newton''s Divided Difference Coefficients:\n');
for k = 1:n_pts
    fprintf('  a_%d = %.6f\n', k-1, dd(1,k));
end

% ---- Interpolating Polynomial Function ----
% P(t) = a0 + a1(t-t0) + a2(t-t0)(t-t1) + ...
% Implemented as nested multiplication (Horner-like)

% Plot the interpolated polynomial
t_fine = linspace(0, 10, 1000);
VT_interp_fine = zeros(size(t_fine));

for k = 1:length(t_fine)
    VT_interp_fine(k) = newtonInterp(t_fine(k), t_data, dd);
end

figure('Name','Part 2 Step 3: VT Interpolation','Color','w',...
       'Position',[100 100 600 400]);
plot(t_fine, VT_interp_fine, 'b-', 'LineWidth', 1.5); hold on;
plot(t_data, VT_data, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on; xlabel('Time (s)'); ylabel('V_T (m/s)');
title('Newton''s Divided Difference Interpolation of V_T');
legend('Interpolated polynomial', 'Data points', 'Location', 'best');
```

```m
% ============================================================
%  PART 2 - STEP 4: SIMULATION WITH VARYING VT
% ============================================================
% Run the PI-controlled simulation with VT varying over time
% using the interpolated polynomial.

p_final = p;
p_final.GC = best_Gc;
p_final.KI = best_KI;

X = zeros(10, N);
X(1:9,1) = x0;
X(10,1)  = 0;

for n = 1:N-1
    % Update VT at each time step using interpolation
    p_final.VT = newtonInterp(t(n), t_data, dd);
    if p_final.VT < 0.01; p_final.VT = 0.01; end

    xn = X(:,n);
    k1 = roverDerivatives_PI(t(n),        xn,           p_final);
    k2 = roverDerivatives_PI(t(n)+dt/2,   xn+(dt/2)*k1,  p_final);
    k3 = roverDerivatives_PI(t(n)+dt/2,   xn+(dt/2)*k2,  p_final);
    k4 = roverDerivatives_PI(t(n)+dt,     xn+dt*k3,      p_final);
    X(:,n+1) = xn + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
end

% Plot results with varying VT
figure('Name','Part 2 Step 4: Response with Varying VT','Color','w',...
       'Position',[50 50 1100 750]);

subplot(2,2,1)
plot(t, X(9,:)*180/pi, 'b', 'LineWidth', 1.5); hold on;
plot(t, 55*ones(size(t)), 'r--', 'LineWidth', 1.2);
grid on; xlabel('Time (s)'); ylabel('\psi (degrees)');
title('Yaw Angle with Varying V_T');
legend('\psi(t)', '\psi_{ref}', 'Location', 'best');

subplot(2,2,2)
plot(t, X(8,:), 'b', 'LineWidth', 1.5);
grid on; xlabel('Time (s)'); ylabel('r (rad/s)');
title('Yaw Rate with Varying V_T');

subplot(2,2,3)
plot(t, X(7,:), 'b', 'LineWidth', 1.5);
grid on; xlabel('Time (s)'); ylabel('v (m/s)');
title('Sway Velocity with Varying V_T');

subplot(2,2,4)
% Plot VT variation over time
VT_plot = zeros(size(t));
for k = 1:N
    VT_plot(k) = newtonInterp(t(k), t_data, dd);
end
plot(t, VT_plot, 'b', 'LineWidth', 1.5); hold on;
plot(t_data, VT_data, 'ro', 'MarkerSize', 6, 'MarkerFaceColor', 'r');
grid on; xlabel('Time (s)'); ylabel('V_T (m/s)');
title('Forward Velocity V_T(t)');
legend('Interpolated', 'Data points', 'Location', 'best');

fprintf('\n===== Part 2 Final Simulation =====\n');
fprintf('Best Gc = %.1f, Best KI = %.1f\n', best_Gc, best_KI);
fprintf('Final heading: %.2f deg\n', X(9,end)*180/pi);
fprintf('Steady-state error: %.2f deg\n', 55 - X(9,end)*180/pi);
fprintf('==================================\n');
```

Figure 5: MATLAB implementation: proportional gain study (Step 1), PI controller with integral state extension (Step 2), Newton's divided difference interpolation of V_T (Step 3), and closed loop simulation with time varying forward velocity (Step 4).

2.5 Conclusions for Part 2

The turning controller for the wheeled rover was successfully redesigned through systematic tuning:

- **Gc tuning:** The original gain $G_C = 11.6$ was found to be too aggressive, producing

sustained oscillations. The optimal value $G_C = 5$ reduces overshoot and settling time significantly, though a steady state error of ≈ 10 remains with proportional only control.

- **PI controller:** Adding an integral term with $K_I = 0.5$ eliminates the steady state error completely, achieving a final heading of exactly 55. The integral was implemented as an additional state variable within the RK4 integration loop.
- **Interpolation:** Newton's Divided Difference method was used to model the time varying forward velocity $V_T(t)$ from six data points. The resulting 5th order polynomial was implemented using Horner's nested form.
- **Varying V_T simulation:** The PI controller demonstrated robustness under time varying conditions, maintaining zero steady state error despite changes in the rover's forward speed.

The final controller design ($G_C = 5$, $K_I = 0.5$) represents a significant improvement over the original proportional controller.